

Pet Territory Wars — Milestone 0 Repository & Development Environment v1.0

Repository modeli: Monorepo

Backend: Go

Mobile: Flutter

Database: PostgreSQL + PostGIS

Local orchestration: Docker Compose

Kapsam: MVP başlangıç altyapısı

1. Amaç

Bu doküman Pet Territory Wars projesinin ilk çalışan geliştirme ortamını tanımlar.

Milestone 0'ın amacı ürün özelliği geliştirmek değildir.

Amaç:

- Repository yapısını kurmak
- Go backend'i ayağa kaldırmak
- Flutter uygulamasını başlatmak
- PostgreSQL + PostGIS ortamını hazırlamak
- Migration altyapısını kurmak
- Docker Compose ile local geliştirme ortamı oluşturmak
- CI kontrollerini başlatmak
- Sonraki milestone'lar için temiz bir temel hazırlamak

Bu milestone tamamlandığında henüz capture, dominance, Walk Engine veya territory sistemi çalışmak zorunda değildir.

Ancak bütün temel process'ler ayağa kalkabilmelidir.

2. Repository Kararı

MVP için monorepo kullanılacaktır.

```
pet-territory-wars/  
├─ backend/  
├─ mobile/  
├─ infrastructure/  
├─ docs/  
├─ scripts/  
└─ .github/
```

```
├─ Makefile
└─ README.md
```

2.1 Neden Monorepo?

Monorepo şu avantajları sağlar:

- Backend ve Flutter değişiklikleri aynı repository içinde takip edilir.
- API sözleşmesi değiştiğinde iki taraf aynı pull request içinde güncellenebilir.
- Docker, migration ve CI dosyaları tek yerde yönetilir.
- Tek issue ve branch sistemi kullanılır.
- Küçük ekip veya tek geliştirici için operasyon daha kolay olur.
- Local ortam tek komutla açılabilir.

Monorepo kullanılmasına rağmen backend ve mobile bağımsız proje olarak kalacaktır.

```
backend/go.mod
mobile/pubspec.yaml
```

birbirinden bağımsızdır.

3. Nihai Başlangıç Dizin Yapısı

```
pet-territory-wars/
├─ backend/
│  └─ cmd/
│     ├── api/
│     │   └─ main.go
│     ├── walk-worker/
│     │   └─ main.go
│     ├── outbox-worker/
│     │   └─ main.go
│     └─ simulator/
│         └─ main.go
│
│  └─ internal/
│     ├── api/
│     ├── application/
│     ├── domain/
│     ├── gameengine/
│     ├── infrastructure/
│     └─ repository/
│
│  └─ migrations/
│  └─ testdata/
└─ go.mod
```

```
|   |   | go.sum
|   |   | └─ Dockerfile
|   |
|   └─ mobile/
|       |   |   | lib/
|       |   |   | |   | app/
|       |   |   | |   | core/
|       |   |   | |   └─ features/
|       |   |   └─ test/
|       |   └─ integration_test/
|       |   └─ pubspec.yaml
|       |   └─ analysis_options.yaml
|       |
|   └─ infrastructure/
|       |   |   | compose.yaml
|       |   |   | docker/
|       |   |   └─ reverse-proxy/
|       |
|   └─ docs/
|       |   |   | architecture/
|       |   |   └─ adr/
|       |
|   └─ scripts/
|       |   |   | migrate.sh
|       |   |   | test.sh
|       |   |   └─ lint.sh
|       |
|   └─ .github/
|       |   |   | workflows/
|       |   |   | |   | backend-ci.yml
|       |   |   | |   └─ mobile-ci.yml
|       |
|   └─ .env.example
|   └─ .gitignore
|   └─ Makefile
|   └─ README.md
```

Milestone 0'da bütün klasörlerin dolu olması gerekmez.

Klasörler yalnızca gerçek kod geldiğinde doldurulur.

4. Repository Kurulumu

Başlangıç komutları:

```
mkdir pet-territory-wars
cd pet-territory-wars
```

```
git init
git branch -M main
```

İlk çalışma branch'i:

```
chore/bootstrap-project
```

Branch oluşturma:

```
git checkout -b chore/bootstrap-project
```

5. Backend Kurulumu

Repository kökünde:

```
mkdir backend
cd backend
```

Go module:

```
go mod init github.com/<organization>/pet-territory-wars/backend
```

Gerçek GitHub organization veya kullanıcı adı oluşturulduğunda `<organization>` değeri güncellenecektir.

6. Go Executable'ları

MVP backend'inde dört executable bulunacaktır.

```
cmd/api
cmd/walk-worker
cmd/outbox-worker
cmd/simulator
```

6.1 API

Görevi:

- REST API'yi çalıştırmak
- Health endpoint'lerini sunmak
- Config yüklemek
- PostgreSQL bağlantısı kurmak
- Structured logging başlatmak

Milestone 0'da yalnızca health endpoint'lerinin çalışması yeterlidir.

6.2 Walk Worker

Görevi:

- İleride `walk_processing_jobs` tablosundan iş almak
- Game Engine'i çalıştırmak
- Walk sonucunu uygulamak

Milestone 0'da yalnızca:

- Config yüklemeli
- Database bağlantısını doğrulamalı
- Process olarak ayağa kalkmalı
- Kontrollü şekilde kapanabilmelidir

Henüz job işlemeyecektir.

6.3 Outbox Worker

Görevi:

- İleride `outbox_events` kayıtlarını işlemek

Milestone 0'da yalnızca boş çalışan process olarak ayağa kalkması yeterlidir.

6.4 Simulator

Görevi:

- Game Engine simülasyonlarını çalıştırmak
- Sahte Walk ve oyuncu senaryoları üretmek
- Calculator Rules değerlerini karşılaştırmak

Milestone 0'da yalnızca executable olarak çalışmalıdır.

7. Backend Başlangıç Paketleri

Milestone 0 için gerekli minimum paketler:

```
internal/infrastructure/config  
internal/infrastructure/logging  
internal/infrastructure/postgres  
internal/api/health
```

İlk aşamada şu paketler boş klasör olarak oluşturulmak zorunda değildir:

```
domain  
gameengine  
repository  
application
```

Gerçek kod yazıldığında eklenir.

8. Config Modeli

Backend config yalnızca teknik değerleri içerir.

```
APP_ENV  
HTTP_PORT  
DATABASE_URL  
LOG_LEVEL  
WALK_WORKER_CONCURRENCY  
OUTBOX_WORKER_CONCURRENCY  
HTTP_READ_TIMEOUT  
HTTP_WRITE_TIMEOUT  
HTTP_SHUTDOWN_TIMEOUT
```

Örnek `.env.example` :

```
APP_ENV=local  
  
HTTP_PORT=8080  
HTTP_READ_TIMEOUT=5s  
HTTP_WRITE_TIMEOUT=10s  
HTTP_SHUTDOWN_TIMEOUT=10s  
  
DATABASE_URL=postgres://pet_app:local_password@postgres:5432/pet_territory?  
sslmode=disable
```

```
LOG_LEVEL=debug
```

```
WALK_WORKER_CONCURRENCY=2  
OUTBOX_WORKER_CONCURRENCY=1
```

Aşağıdaki değerler environment config içinde tutulmaz:

```
H3 resolution  
Maximum dominance  
Attack damage  
Daily decay  
Minimum Walk distance
```

Bunlar ileride RuleSet üzerinden yönetilecektir.

9. Config Validation

Application aşağıdaki durumlarda başlamamalıdır:

- `DATABASE_URL` boşsa
- HTTP port geçersizse
- Worker concurrency sıfırdan küçükse
- Environment desteklenmeyen bir değerse
- Timeout değerleri parse edilemiyorsa

Desteklenen environment değerleri:

```
local  
staging  
production  
test
```

Config hataları process başlangıcında açıkça gösterilmelidir.

10. Structured Logging

Bütün Go process'leri stdout'a structured JSON log yazmalıdır.

Örnek:

```
{  
  "level": "info",  
  "message": "api started",
```

```
"service": "api",  
"environment": "local",  
"port": 8080  
}
```

Ortak alanlar:

```
service  
environment  
version  
timestamp  
level  
message
```

Milestone 0'da raw GPS veya domain verisi bulunmadığından hassas veri riski sınırlıdır.

Ancak logging temeli baştan güvenli kurulmalıdır.

11. Graceful Shutdown

API ve worker process'leri aşağıdaki sinyalleri yakalamalıdır:

```
SIGINT  
SIGTERM
```

Kapanış sırasında:

- Yeni HTTP isteği kabul edilmez.
- Aktif isteklerin tamamlanması için timeout uygulanır.
- Database pool kapatılır.
- Worker yeni job almayı bırakır.
- Log buffer varsa flush edilir.

Container'ın zorla öldürülmesine güvenilmez.

12. Health Endpoint'leri

API şu endpoint'leri sunacaktır:

```
GET /health/live  
GET /health/ready
```


12.1 Liveness

```
GET /health/live
```

Response:

```
{  
  "status": "ok"  
}
```

Bu endpoint yalnızca process'in çalıştığını gösterir.

Database kontrol etmez.

12.2 Readiness

```
GET /health/ready
```

Bu endpoint:

- Config'in geçerli olduğunu
- PostgreSQL bağlantısının çalıştığını

kontrol eder.

Başarılı response:

```
{  
  "status": "ready"  
}
```

Database erişilemiyorsa:

```
503 Service Unavailable
```

dönmelidir.

13. Flutter Kurulumu

Repository kökünde:

```
flutter create
  --org com.petterritory
  --project-name pet_territory
  mobile
```

Milestone 0'da kurulacak temel yapı:

```
lib/app
lib/core
lib/features
```

Başlangıç bağımlılıkları:

```
Riverpod
Router
HTTP client
Secure storage abstraction
SQLite altyapısı
Environment config
```

GPS ve harita package'ları henüz eklenmeyebilir.

Bu bağımlılıklar ilgili milestone'da seçilecektir.

14. Flutter Başlangıç Ekranı

Milestone 0'da Flutter uygulamasının yalnızca açılması ve temel environment bilgisini okuyabilmesi yeterlidir.

Örnek ekran:

```
Pet Territory Wars
Environment: local
API status: connected
```

API health endpoint'i çağrılarak backend bağlantısı doğrulanabilir.

Bu production ekranı değildir; bootstrap doğrulama ekranıdır.

15. Flutter Environment Yapısı

En az şu environment'lar desteklenmelidir:

```
local
staging
production
```

Mobil config örnekleri:

```
API_BASE_URL
APP_ENV
LOG_LEVEL
```

Business rule mobil config'e eklenmez.

Flutter authoritative RuleSet saklamaz.

16. PostgreSQL + PostGIS

Local geliştirme ortamı için PostGIS içeren PostgreSQL image kullanılacaktır.

Örnek Compose servisi:

```
services:
  postgres:
    image: postgis/postgis:16-3.4
    environment:
      POSTGRES_DB: pet_territory
      POSTGRES_USER: pet_app
      POSTGRES_PASSWORD: local_password
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test:
        [
          "CMD-SHELL",
          "pg_isready -U pet_app -d pet_territory"
        ]
      interval: 5s
      timeout: 5s
      retries: 10
```

Image sürümü sabitlenmelidir.

`latest` kullanılmamalıdır.

17. Docker Compose Servisleri

Milestone 0 Compose yapısı:

```
postgres
api
walk-worker
outbox-worker
```

Reverse proxy local geliřtirmede zorunlu deęildir.

Local API:

```
http://localhost:8080
```

üzerinden çalışabilir.

18. Dockerfile Yaklaşımı

Go servisleri multi-stage build kullanacaktır.

```
Build stage
  ↓
Go binary
  ↓
Minimal runtime image
```

Tercih:

```
distroless
```

veya gerekli sertifikalar açıkça yönetilebiliyorsa:

```
scratch
```

Milestone 0 için distroless daha kolay ve güvenli bir başlangıçtır.

API, Walk Worker, Outbox Worker ve Simulator aynı backend kaynak kodundan farklı command'larla build edilir.

19. Tek Dockerfile Kararı

MVP'de dört ayrı Dockerfile oluşturmak yerine tek backend Dockerfile kullanılabilir.

Build argument:

```
TARGET=api
TARGET=walk-worker
TARGET=outbox-worker
TARGET=simulator
```

Örnek:

```
docker build
  --build-arg TARGET=api
  -t pet-territory-api .
```

Bu yaklaşım gereksiz Dockerfile tekrarını önler.

20. Migration Altyapısı

Migration application başlangıcında otomatik çalıştırılmaz.

Ayrı komut veya container ile çalıştırılır.

Migration aracı olarak aşağıdakilerden biri seçilebilir:

```
golang-migrate
Goose
Atlas
```

MVP için öneri:

```
golang-migrate
```

Sebebi:

- Basit
 - Yaygın
 - SQL-first
 - Uygulamadan bağımsız
 - CI ve Docker içinde kolay kullanım
-

21. İlk Migration'lar

21.1 PostGIS Extension

Dosya:

```
000001_enable_postgis.up.sql
```

İçerik:

```
CREATE EXTENSION IF NOT EXISTS postgis;
```

Down migration:

```
000001_enable_postgis.down.sql
```

Production'da extension drop etmek tehlikeli olacağı için boş bırakılabilir veya kontrollü yönetilebilir.

21.2 İlk Şema Tabloları

Core entity tabloları Milestone 2'de oluşturulacaktır.

Milestone 0'da şema yalnızca migration altyapısının çalıştığını kanıtlamalıdır.

Gereksiz baseline tablosu oluşturulmaz; migration aracı kendi version takibini yapar.

22. Makefile

Repository kökünde Makefile bulunacaktır.

Önerilen komutlar:

```
make up
make down
make restart
make logs
make ps
make migrate-up
make migrate-down
make migrate-create
make backend-test
```

```
make backend-lint
make backend-build
make mobile-test
make mobile-lint
make simulator
```

Örnek Kullanım

```
make up
make migrate-up
make backend-test
```

Komut isimleri kısa ama açık olmalıdır.

23. Scripts

Shell script'ler yalnızca Makefile'ın karmaşıklaşmasını önlemek için kullanılmalıdır.

Önerilen script'ler:

```
scripts/migrate.sh
scripts/test.sh
scripts/lint.sh
```

Script kuralları:

```
set -euo pipefail
```

kullanmalıdır.

Hata durumunda sessizce devam etmemelidir.

24. Backend CI

Her pull request'te:

```
gofmt check
go vet
staticcheck
golangci-lint
go test
```

```
go test -race
govulncheck
go build
```

çalışmalıdır.

Integration testler daha sonra PostgreSQL service container ile eklenecektir.

Milestone 0'da en azından build ve temel test pipeline'ı çalışmalıdır.

25. Flutter CI

Her pull request'te:

```
dart format check
flutter analyze
flutter test
```

çalışmalıdır.

Android ve iOS production build işlemleri Milestone 0'da zorunlu değildir.

Ancak Flutter project build edilebilir durumda olmalıdır.

26. Git Ignore

`.gitignore` en az şu dosyaları kapsamalıdır:

```
.env
.env.*
!.env.example

backend/bin/
backend/coverage.out

mobile/.dart_tool/
mobile/build/
mobile/.flutter-plugins
mobile/.flutter-plugins-dependencies

.idea/
.vscode/
.DS_Store
```



```
*.log
```

Production secret veya local database dosyası commit edilmemelidir.

27. README

Repository ana README'si şu başlıkları içermelidir:

```
Project overview
Architecture summary
Requirements
Local installation
Environment variables
Docker commands
Migration commands
Backend tests
Flutter tests
Common errors
```

Yeni geliştirici README üzerinden projeyi ayağa kaldırabilmelidir.

28. Local Geliştirme Akışı

Önerilen akış:

```
git clone <repository>
cd pet-territory-wars

cp .env.example .env

make up
make migrate-up
```

Doğrulama:

```
curl http://localhost:8080/health/live
curl http://localhost:8080/health/ready
```

Flutter:

```
cd mobile
flutter pub get
flutter run
```

29. Database Connection Pool

Milestone 0 başlangıç değerleri düşük tutulmalıdır.

Örnek:

```
API max open connections: 10
Walk Worker: 5
Outbox Worker: 2
```

Local ve production değerleri aynı olmak zorunda değildir.

Pool değerleri config üzerinden yönetilir.

Daha yüksek connection sayısı otomatik olarak daha yüksek performans anlamına gelmez.

30. Service Restart Politikası

Docker Compose içinde process'ler için:

```
restart: unless-stopped
```

kullanılabilir.

Ancak sürekli crash eden process'in sadece restart edilmesi yeterli değildir.

Loglarda hata görünür olmalı ve health check başarısız olmalıdır.

31. Version Bilgisi

Backend binary build sırasında şu bilgiler eklenebilir:

```
version
git commit
build time
```

Health veya internal info logunda gösterilebilir.

Production API response'unda gereksiz ayrıntı sunulmaz.

32. Milestone 0 Kapsam Dışı

Bu aşamada yapılmayacaklar:

```
Game Engine kuralları
Domain entity'leri
Core database tabloları
Authentication
Walk API
GPS upload
Walk Worker job processing
Outbox event processing
Map
Leaderboard
Redis
Reverse proxy production config
Kubernetes
Terraform
Push notification
```

Milestone 0'nın amacı bunlara sağlam temel hazırlamaktır.

33. Uygulama Sırası

Milestone 0 şu sırayla uygulanacaktır:

1. Monorepo oluştur
2. Root dosyaları ekle
3. Go module oluştur
4. Dört Go executable ekle
5. Config loader oluştur
6. Structured logger oluştur
7. PostgreSQL connection pool oluştur
8. Health endpoint'leri oluştur
9. Dockerfile oluştur
10. Compose oluştur

11. Migration altyapısı oluştur
12. Flutter project oluştur
13. Flutter environment temelini oluştur
14. Makefile oluştur
15. Backend CI oluştur
16. Flutter CI oluştur
17. README yaz
18. Local kurulum testini sıfırdan çalıştır

34. Definition of Done

Milestone 0 tamamlanmış sayılırsa:

1. Monorepo oluşturulmuş olmalı.
2. Go module çalışmalı.
3. API executable ayağa kalkmalı.
4. Walk Worker executable ayağa kalkmalı.
5. Outbox Worker executable ayağa kalkmalı.
6. Simulator executable çalışmalı.
7. Flutter uygulaması açılmalı.
8. PostgreSQL + PostGIS çalışmalı.
9. Migration komutu başarılı olmalı.
10. Health live endpoint'i çalışmalı.
11. Health ready PostgreSQL'i doğrulamalı.
12. Structured JSON log üretilmeli.
13. Graceful shutdown çalışmalı.
14. Docker Compose tek komutla ortamı açmalı.
15. Environment validation çalışmalı.
16. Backend CI başarılı olmalı.
17. Flutter CI başarılı olmalı.
18. Secret'lar repository'de bulunmamalı.
19. README ile sıfırdan kurulum yapılabilmesi.
20. Business rule veya gereksiz özellik eklenmemiş olmalı.

35. Nihai Karar

Pet Territory Wars MVP başlangıç yapısı:

Monorepo
Go backend
Flutter mobile
PostgreSQL + PostGIS
Docker Compose
Dört bağımsız Go executable

SQL migration
Structured logging
Health checks
CI

üzerine kurulacaktır.

Temel hedef:

Projenin ilk özelliğini yazmadan önce, backend, mobile, database ve geliştirme araçlarının tekrarlanabilir ve güvenilir biçimde çalıştığı bir temel oluşturmak.

Milestone 0 tamamlandıktan sonra sıradaki aşama:

Milestone 1 — Calculator Engine & Simulator implementasyonu